

幻思创新——VINS-ROS-EGO 工程开发指南

注：如遇无法解决的问题，请联系幻思创新的工程师寻求协助。

修订日志：

版本	日期	更新内容
V2.1.3	2025.1.20	1、新增“启用 VINS 定位”指南。（P18， 点击跳转 ） 2、新增“启用 EGO 和下发目标点”指南。（P24， 点击跳转 ）
V2.1.4	2025.3.26	1、更新了“启用 VINS（VSLAM）定位”章节中的“固件版本号要求”和“配置飞控固件参数”。（P18，P19， 点击跳转 ）
V2.1.5	2025.4.7	1、更新了“开机自启动”章节中的“（4）配置 auto_ap.sh 开机自动运行”。（P17， 点击跳转 ）
V2.1.6	2025.5.7	1、更新了“启用 VINS（VSLAM）定位”章节中的“1、配置飞控固件参数”——>波特率设置。（P19， 点击跳转 ） 2、增加了“启用 VINS（VSLAM）定位”章节中的“3、PID 调参”。（P20， 点击跳转 ）
V2.1.7	2025.5.25	1、V1.2.7 版本在“机载电脑 fcu_core 工程”、“安装以及配置 cuda”、“配置 realsense（深度相机相关驱动）”、“全部工程自动运行”、“开机自启动”及“启用 VINS（VSLAM）定位”章节均有更新。
V2.1.8	2025.8.16	1、更新了“ROS1 安装”章节，增加了说明，对其中的“（1）配置 rosdep”进行了完善。（P2， 点击跳转 ）

V2.1.9	2025.10.9	1、更新了“启用 VINS (VSLAM) 定位” 章节，对其中的“2、烧录固件”进行了完善。（P21， 点击跳转 ）
V2.2.1	2025.11.22	1、更新了“配置 realsense（深度相机相关驱动）” 章节，对其中的“6 和 7”进行了完善。（P10， 点击跳转 ）
V2.2.2	2025.11.27	1、更新了“配置 realsense（深度相机相关驱动）” 章节，对其中的“6”进行了完善。（P10， 点击跳转 ）
V2.2.3	2026.3.12	1、将“配置 RealSense（深度相机相关驱动）”章节移动至“配置 cv_bridge 功能包”章节之后。（P12， 点击跳转 ） 2、同时对“开机自启动”章节进行了更新，并完善了结尾部分的相关说明。（P20， 点击跳转 ）
V2.2.4	2026.7.15	1、新增“不启用 VINS (VSLAM) 定位”章节，并完善了无人机下视光流定位说明。（P20， 点击跳转 ）

系统版本

UBUNTU 版本 20.04

ROS1 安装

说明：① 系统已预装 ROS1，无需重复安装，可直接跳过 ROS1 安装步骤。

② 如何确认系统已安装 ROS1：

第一步：打开终端，输入命令：**roscore**

第二步：查看运行结果，如下图：



```
ubuntu20@ubuntu:~/ros_ws$ roscore
... logging to /home/ubuntu20/.ros/log/166c17d0-78e9-11f0-bd76-adcb4b38d7b6/roslaunch-ubuntu-8493.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu:32793/
ros_comm version 1.17.4

SUMMARY
=====
PARAMETERS
 * /rostdistro: noetic
 * /rosversion: 1.17.4

NODES
auto-starting new master
process[master]: started with pid [8501]
ROS_MASTER_URI=http://ubuntu:11311/

setting /run_id to 166c17d0-78e9-11f0-bd76-adcb4b38d7b6
process[/rosout-1]: started with pid [8511]
started core service [/rosout]
```

(1)配置源

终端输入：**wget http://fishros.com/install -O fishros && . fishros**

然后会出现以下界面：

```
RUN Choose Task:[请输入括号内的数字]
---众多工具，等君来用---
ROS相关:
[1]:一键安装(推荐):ROS(支持ROS/ROS2,树莓派Jetson)
[4]:一键配置:ROS环境(快速更新ROS环境设置,自动生成环境选择)
[3]:一键安装:rosdep(小鱼的rosdep,又快又好用)
[9]:一键安装:Cartographer(内测版易失败)
[11]:一键安装:ROS Docker版(支持所有版本ROS/ROS2)

常用软件:
[2]:一键安装:github桌面版(小鱼常用的github客户端)
[6]:一键安装:NodeJS环境
[7]:一键安装:VsCode开发工具
[8]:一键安装:Docker
[10]:一键安装:微信(可以在Linux上使用的微信)
[12]:一键安装:PlatformIO MicroROS开发环境(支持Fishbot)
[14]:一键安装:科学上网代理工具
[15]:一键安装:QQ for Linux

配置工具:
[5]:一键配置:系统源(更换系统源,支持全版本Ubuntu系统)
[13]:一键配置:python国内源

[0]:quit

请输入[]内的数字以选择:█
```

接着，输入数字 5，回车，一键配置系统源

```
欢迎使用一键更换系统源,本工具由作者小鱼提供
欢迎使用一键换源工具,本工具由[鱼香ROS]小鱼贡献..
RUN Choose Task:[请输入括号内的数字]
请选择换源方式,如果不知道选什么请选2
[1]:仅更换系统源
[2]:更换系统源并清理第三方源
[0]:quit
请输入[]内的数字以选择:█
```

输入数字 2，回车，更换系统源并清理第三方源

```
RUN Choose Task:[请输入括号内的数字]
请问是否添加ROS和ROS2源?
[1]:添加ROS/ROS2源
[2]:不添加ROS/ROS2源
[0]:quit
请输入[]内的数字以选择:█
```

输入数字 1，回车，添加 ROS/ROS2 源

(2)安装 ROS

终端输入：`wget http://fishros.com/install -O fishros && . fishros`

然后会出现以下界面：

```
RUN Choose Task:[请输入括号内的数字]
---众多工具，等君来用---
ROS相关:
[1]:一键安装(推荐):ROS(支持ROS/ROS2,树莓派Jetson)
[4]:一键配置:ROS环境(快速更新ROS环境设置,自动生成环境选择)
[3]:一键安装:rosdep(小鱼的rosdep,又快又好用)
[9]:一键安装:Cartographer(内测版易失败)
[11]:一键安装:ROS Docker版(支持所有版本ROS/ROS2)

常用软件:
[2]:一键安装:github桌面版(小鱼常用的github客户端)
[6]:一键安装:NodeJS环境
[7]:一键安装:VsCode开发工具
[8]:一键安装:Docker
[10]:一键安装:微信(可以在Linux上使用的微信)
[12]:一键安装:PlatformIO MicroROS开发环境(支持Fishbot)
[14]:一键安装:科学上网代理工具
[15]:一键安装:QQ for Linux

配置工具:
[5]:一键配置:系统源(更换系统源,支持全版本Ubuntu系统)
[13]:一键配置:python国内源

[0]:quit

请输入[]内的数字以选择:█
```

然后我们输入 1 一键安装 → 不更换源安装 → 选择你 ubuntu 版本对应的 ros 版本(这里我们选择 ROS1 的 noetic) → 桌面版进行安装

(3)配置 rosdep

```
source ~/.bashrc
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator python3-wstool build-essential
sudo rosdep init
rosdep update
```

到这里已完成 ROS1 安装。

安装超级终端

便于同时运行多个程序，可以进行横向纵向分割

```
sudo apt-get install terminator -y
```

机载电脑 fcu_core 工程

开启新终端：

- (1) `mkdir ros_ws`
- (2) `cd ros_ws`
- (3) `mkdir src`
- (4) `cd src`
- (5) `git clone https://gitee.com/fancinnov/fcu\_core.git`
- (6) `git clone https://gitee.com/fancinnov/quadrotor\_msgs.git`
- (7) `sudo apt-get install ros-noetic-serial`
- (8) `sudo apt-get install libeigen3-dev`
- (9) 修改 fcu_core 工程的 fcu_bridge_001.cpp 文件，注意可以直接用幻思提供的已修改好的文件替换原工程文件，原工程路径为 ros_ws/src/fcu_core/src。
- (10) 在工作空间：`catkin_make`

如果用幻思提供的已配好的 fcu_bridge_001.cpp 文件进行替换后，可以忽略下述配置。

Fcu_bridge_001.cpp 文件

1. 参数配置：

```
24 #define BUF_SIZE 32768 //数据缓存区大小
25 #define BAUDRATE 460800 //虚拟串口波特率
26 #define DRONE_PORT 333 //port
27 static char* DRONE_IP = "192.168.0.201"; //ip
28 static char* USB_PORT = "/dev/ttyTH50"; //usb虚拟串口文件描述符
29 static mavlink_channel_t mav_chan=MAVLINK_COMM_0; //MAVLINK_COMM_0虚拟串口发送，MAVLINK_COMM_1网口发送
30 static bool offboard=true; //是否使用机载电脑
31 static bool use_uwb=false; //是否使用UWB基站
32 static bool set_goal=false; //远程电脑用于设置轨迹规划的目标，机载电脑应为false
33 static bool simple_target=true; //仅机载电脑配置：是否为简单目标点，simple_target表示目标只有位置，没有速度和加速度
34
```

`static bool simple_target=true;` // true 表示目标只有位置，没有速度和加速度

2. IMU 坐标对齐：

```

239
240
241
242
243
244
245
246
247
248
249
250
251
252
253

```

```

case MAVLINK_MSG_ID_SCALED_IMU:
    mavlink_msg_scaled_imu_decode(&msg_received, &imu);
    imu_pub.header.frame_id = "scaled_imu";
    imu_pub.header.stamp = ros::Time::now();
    imu_pub.linear_acceleration.x = (double)imu.yacc/1000;
    imu_pub.linear_acceleration.y = (double)imu.zacc/1000;
    imu_pub.linear_acceleration.z = (double)imu.xacc/1000;
    imu_pub.angular_velocity.x = (double)imu.ygyro/1000;
    imu_pub.angular_velocity.y = (double)imu.zgyro/1000;
    imu_pub.angular_velocity.z = (double)imu.xgyro/1000;
    imu_global.publish(imu_pub);
    // printf("acc0,acc1,acc2,gyr0,gyr1,gyr2: %f %f %f %f %f %f\n",
    //         (double)imu.xacc/1000,-(double)imu.yacc/1000,-(double)imu.zacc/1000,
    //         (double)imu.xgyro/1000,-(double)imu.ygyro/1000,-(double)imu.zgyro/1000);
    break;

```

原始的 y,z,x 对应于发布的 x,y,z;

3. Ego 里程计坐标对齐:

```

265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290

```

```

case MAVLINK_MSG_ID_GLOBAL_VISION_POSITION_ESTIMATE:
    mavlink_msg_global_vision_position_estimate_decode(&msg_received, &pose);
    odom_pub.header.frame_id = "map";
    odom_pub.header.stamp = ros::Time::now();
    float quaternion_odom[4];
    mavlink_euler_to_quaternion(pose.roll, -pose.pitch, -pose.yaw, quaternion_odom);
    odom_pub.pose.pose.orientation.w=quaternion_odom[0];
    odom_pub.pose.pose.orientation.x=quaternion_odom[1];
    odom_pub.pose.pose.orientation.y=quaternion_odom[2];
    odom_pub.pose.pose.orientation.z=quaternion_odom[3];
    odom_pub.pose.pose.position.x=pose.x*0.01;
    odom_pub.pose.pose.position.y=-pose.y*0.01;
    odom_pub.pose.pose.position.z=pose.z*0.01;
    if (use_ublox(position.lat==0||position.lon==0))
        break;
    }
    odomPose.header = odom_pub.header;
    odomPose.pose = odom_pub.pose.pose;
    path_pub.header.stamp = odom_pub.header.stamp;
    path_pub.poses.push_back(odomPose);
    path_pub.header.frame_id = "map";

    odom_global.publish(odom_pub);
    path_global.publish(path_pub);
    break;

```

姿态和位置都要对齐

4. VINS 里程计坐标系对齐:

```

322 void odomHandler(const nav_msgs::Odometry::ConstPtr& odom)
323 {
324     if(time_odom<time_start||nav_chan == MAVLINK_COMM_0){//USB传输无需降频
325         time_odom=ros::Time::now().toSec();
326     }else{
327         if(ros::Time::now().toSec()-time_odom<0.05){//用网络通信, 如果odom频率过高进行降频, 降至20hz以下
328             return;
329         }
330         time_odom=ros::Time::now().toSec();
331     }
332 }
333 Eigen::Vector3f position_map; ((float)odom->pose.pose.position.y, (float)odom->pose.pose.position.x, -(float)odom->pose.pose.position.z);
334 float quaternion_odom[4]=((float)odom->pose.pose.orientation.w,
335 (float)odom->pose.pose.orientation.x,
336 (float)odom->pose.pose.orientation.y,
337 (float)odom->pose.pose.orientation.z);
338
339 float roll, pitch, yaw;
340 mavlink_quaternion_to_euler(quaternion_odom, &roll, &pitch, &yaw);
341 printf("x:%f,y:%f,z:%f,yaw:%f\n",position_map.x(),position_map.y(),position_map.z(),-yaw);
342
343 mavlink_message_t msg_local_position_ned_cov, msg_attitude;
344 mavlink_attitude_t attitude;
345 mavlink_local_position_ned_cov_t local_position_ned_cov;
346
347 attitude.yaw = -yaw;
348 mavlink_msg_attitude_encode(mavlink_system.sysid, mavlink_system.compid, &msg_attitude, &attitude);
349 mavlink_send_msg(nav_chan, &msg_attitude);
350
351 local_position_ned_cov.x=position_map.x();
352 local_position_ned_cov.y=position_map.y();
353 local_position_ned_cov.z=0.0f;
354 mavlink_msg_local_position_ned_cov_encode(mavlink_system.sysid, mavlink_system.compid, &msg_local_position_ned_cov, &local_position_ned_cov);
355 mavlink_send_msg(nav_chan, &msg_local_position_ned_cov);
356 }

```

原始的 x,y,z 对应于 y,x,-z; 原始的 yaw 对应于-yaw;

5. ROS 话题调整:

```
410 gnss_global = nh.advertise<sensor_msgs::NavSatFix>("gnss_global_001",100);
411 imu_global = nh.advertise<sensor_msgs::Imu>("imu_global_001",100);
412 odom_global = nh.advertise<nav_msgs::Odometry>("odom_global_001",100);
413 odom=nh.subscribe<nav_msgs::Odometry>("/loop_fusion/odometry_rect", 100, odomHandler);
414 cmd=nh.subscribe<std_msgs::Int16>("/fcu_bridge/command", 100, cmdHandler);
415 mission=nh.subscribe<geometry_msgs::InertiaStamped>("/fcu_bridge/mission_001", 100, missionHandler);
416 path_global = nh.advertise<nav_msgs::Path>("/path_global_001", 100);
417 goal=nh.advertise<geometry_msgs::PoseStamped>("/move_base_simple/goal", 100);
418
```

安装以及配置 cuda

1. 添加源

```
sudo gedit /etc/apt/sources.list.d/nvidia-l4t-apt-source.list
```

加下面两行进去

```
deb http://repo.download.nvidia.com/jetson/common r35.4 main
```

```
deb http://repo.download.nvidia.com/jetson/t234 r35.4 main
```

2. 更新源

```
sudo apt update
```

3. 装 jetson 包

```
sudo apt install nvidia-jetpack
```

4. 配置环境变量

```
gedit ~/.bashrc
```

下面添加：

```
export LD_LIBRARY_PATH=/usr/local/cuda/lib64
```

```
export PATH=/usr/local/cuda/bin:$PATH
```

```
export CUDA_HOME=/usr/local/cuda
```

5. 更新环境变量

```
source ~/.bashrc
```

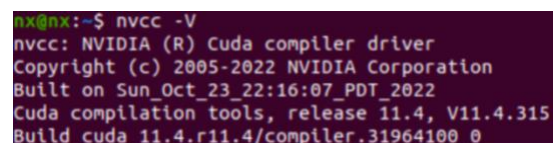
6. 复制文件到 cuda 目录

```
cd /usr/include && sudo cp cudnn* /usr/local/cuda/include
```

```
cd /usr/lib/aarch64-linux-gnu && sudo cp libcudnn* /usr/local/cuda/lib64
```

7. 查看 cuda

nvcc -V (出现下述信息代表安装 cuda 成功)



```
nx@nx:~$ nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Sun_Oct_23_22:16:07_PDT_2022
Cuda compilation tools, release 11.4, V11.4.315
Build cuda_11.4.r11.4/compiler.31964100_0
```

8. 安装 jtop (便于查询系统情况)

```
sudo apt install python3-pip
```

```
sudo -H pip3 install -U pip
```

```
sudo -H pip install jetson-stats
```

使用如下指令运行

```
jtop
```

安装 opencv4.6.0 with cuda 以支持 vins 的 GPU 加速

下载地址: [Release OpenCV 4.6.0 · opencv/opencv](https://github.com/opencv/opencv/releases/tag/4.6.0)

扩展模块: [Release 4.6.0 · opencv/opencv_contrib](https://github.com/opencv/opencv_contrib/releases/tag/4.6.0)

(如果网不好, 这两个依赖库也可以用幻思提供的)

解压放在 home 路径即可

确定 Jetson Orin NX 的算力为 8.7

Jetson Products

GPU	Compute Capability
Jetson AGX Orin, Jetson Orin NX, Jetson Orin Nano	8.7
Jetson AGX Xavier, Jetson Xavier NX	7.2
Jetson TX2	6.2
Jetson Nano	5.3

CSDN @想要个小姑娘

```
cd opencv-4.6.0/
```

```
mkdir build && cd build (如果用幻思的库, 已经存在 build 文件夹, 只需要 cd build)
```

预编译 opencv 4.6.0 及其扩展模块 opencv_contrib-4.6.0, 生成 Makefiles 文件

```
cmake -D CMAKE_BUILD_TYPE=RELEASE \
-D CMAKE_INSTALL_PREFIX=/usr/local/ \
-D OPENCV_EXTRA_MODULES_PATH=../../opencv_contrib-4.6.0/modules \
-D WITH_CUDA=ON \
-D CUDA_ARCH_BIN=8.7 \
-D CUDA_ARCH_PTX="" \
-D ENABLE_FAST_MATH=ON \
-D CUDA_FAST_MATH=ON \
-D WITH_CUBLAS=ON \
-D WITH_LIBV4L=ON \
-D WITH_GSTREAMER=ON \
-D WITH_GSTREAMER_0_10=OFF \
-D WITH_QT=ON \
-D WITH_OPENGL=ON \
-D CUDA_NVCC_FLAGS="--expt-relaxed-constexpr" \
-D WITH_TBB=ON \
..
```

其中

CMAKE_INSTALL_PREFIX=/usr/local/ 为安装地址，

OPENCV_EXTRA_MODULES_PATH=../../opencv_contrib-4.6.0/modules 为扩展模块所在路径，

CUDA_ARCH_BIN=8.7 为 GPU 算力，

编译完成后如下所示

```
--
-- Python 3:
-- Interpreter:           /usr/bin/python3 (ver 3.8.10)
-- Libraries:            /usr/lib/aarch64-linux-gnu/libpython3.8.so
ver 3.8.10)
-- numpy:                /usr/lib/python3/dist-packages/numpy/core/l
clude (ver 1.17.4)
-- Install path:         lib/python3.8/site-packages/cv2/python-3.8
--
-- Python (for build):    /usr/bin/python2.7
--
-- Java:
-- ant:                  NO
-- JNI:                  NO
-- Java wrappers:        NO
-- Java tests:           NO
--
-- Install to:            /usr/local
-----
-- Configuring done
-- Generating done
```

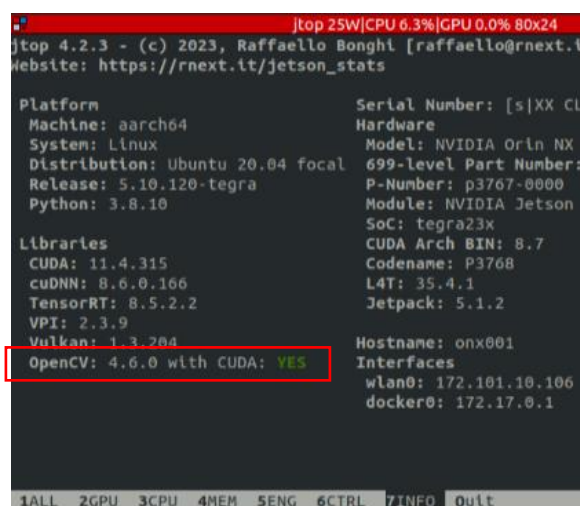
然后 make install 编译安装 opencv 4.6.0 及其扩展模块 opencv_contrib-4.6.0

sudo make install -j8

安装完成后用

jtop

查看版本



```
jtop 25W|CPU 6.3%|GPU 0.0% 80x24
jtop 4.2.3 - (c) 2023, Raffaello Bonghi [raffaello@rnext.it]
Website: https://rnext.it/jetson_stats

Platform                               Serial Number: [s]XX CL
Machine: aarch64                       Hardware
System: Linux                           Model: NVIDIA Orin NX
Distribution: Ubuntu 20.04 focal         699-level Part Number:
Release: 5.10.120-tegra                 P-Number: p3767-0000
Python: 3.8.10                          Module: NVIDIA Jetson
                                           SoC: tegra23x
Libraries                               CUDA Arch BIN: 8.7
CUDA: 11.4.315                          Codename: P3768
cuDNN: 8.6.0.166                        L4T: 35.4.1
TensorRT: 8.5.2.2                       Jetpack: 5.1.2
VPI: 2.3.9
Vulkan: 1.3.264                          Hostname: onx001
OpenCV: 4.6.0 with CUDA: YES             Interfaces
                                           wlan0: 172.101.10.106
                                           docker0: 172.17.0.1

1ALL 2GPU 3CPU 4MEM 5ENG 6CTRL 7INFO Quit
```

配置 cv_bridge 功能包

创建一个 ROS 的工作空间

```
mkdir -p ~/catkin_pkg/src/
```

```
cd ~/catkin_pkg/src/
```

下载对应版本的源码（如果网不好，这个功能包也可以用幻思提供的）

```
git clone https://github.com/ros-perception/vision_opencv.git -b noetic
```

```
cd ~/catkin_pkg/src/vision_opencv/cv_bridge/
```

```
gedit ./CMakeLists.txt
```

在 CMakeLists.txt 文件添加自己安装的带 GPU 加速的 OpenCV 的 build 文件的路径（幻思提供的已经配置好了，直接跳过此步进行编译即可）。

```
include("${~/opencv-4.6.0/build/OpenCVConfig.cmake}")
```

```
9  if(PYTHONLIBS_VERSION_STRING VERSION_LESS "3.8")
10     # Debian Buster
11     find_package(Boost REQUIRED python37)
12  else()
13     # Ubuntu Focal
14     find_package(Boost REQUIRED python)
15  endif()
16 else()
17  find_package(Boost REQUIRED)
18 endif()
19
20 include("${~/Documents/opencv-4.6.0/build/OpenCVConfig.cmake}")
21 set(_opencv_version 4)
22 find_package(OpenCV 4 QUIET)
23 if(NOT OpenCV_FOUND)
24  message(STATUS "Did not find OpenCV 4, using OpenCV 3")
25  set(_opencv_version 3)
26 endif()
```

保存后编译该功能包，终端输入

```
cd ~/catkin_pkg/
```

```
catkin_make
```

编译完成后将其路径添加到 ~/.bashrc 文件中，并刷新环境变量

```
echo "source ~/catkin_pkg/devel/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

配置 realsense（深度相机相关驱动）

1. clone 源码

```
git clone https://github.com/IntelRealSense/librealsense.git
```

如果下载失败可以用幻思库里的

2. 安装所需的包以及编译源码

```
cd librealsense
```

```
sudo apt-get update
```

```
sudo apt-get install git cmake libssl-dev libusb-1.0-0-dev pkg-config libgtk-3-dev
```

```
sudo apt-get install libglfw3-dev libgl1-mesa-dev libglu1-mesa-dev
```

mkdir build（如果用幻思提供的软件包，这个 build 已经存在，跳过此步）

```
cd build
```

```
cmake ../ (这一步如果网不好多试几次)
```

```
make -j8
```

```
sudo make install
```

3. 配置 USB 规则

```
cd ..
```

```
sudo cp config/99-realsense-libusb.rules /etc/udev/rules.d/
```

```
sudo udevadm control --reload-rules
```

```
sudo udevadm trigger
```

4. 验证安装

```
rs-enumerate-devices    #成功的话会列出设备信息
```

```
realsense-viewer        #查看视频流
```

5. 安装对应的 python 包

```
pip install pyrealsense2
```

6. 编译 realsense 的 ros 工程

```
sudo apt-get install ros-noetic-ddynamic-reconfigure
```

```
cd realsense_ws
```

```
catkin_make
```

7. 启动相机 ros 工程：

```
source devel/setup.bash
```

```
roslaunch realsense2_camera rs_camera.launch
```

VINS-Fusion-GPU 的编译和运行

编译安装 ceres-solver-1.14.0:

```
wget ceres-solver.org/ceres-solver-1.14.0.tar.gz
```

```
tar -zxvf ceres-solver-1.14.0.tar.gz
```

```
# google-glog + gflags
```

```
sudo apt-get install libgoogle-glog-dev libgflags-dev
```

```
# Use ATLAS for BLAS & LAPACK
```

```
sudo apt-get install libatlas-base-dev
```

```
# Eigen3
```

```
sudo apt-get install libeigen3-dev
```

```
cd ~/ceres-solver-1.14.0
```

```
mkdir build && cd build （如果用幻思的库，已经存在 build 文件夹，只需要 cd build）
```

```
cmake ..
```

```
sudo make install -j8
```

编译安装 vin-fusion-gpu

```
#创建工作空间
```

```
mkdir -p ~/vins-fusion-gpu/src/
```

```
cd ~/vins-fusion-gpu/src/
```

如果直接下载幻思库中的 vin-fusion-gpu 无需进行下面配置，可以直接编译

```
cd ~/vins-fusion-gpu/
```

```
catkin_make
```

```
#clone 源码
```

```
git clone https://github.com/pjrambo/VINS-Fusion-gpu.git
```

```
#修改 vins_estimator/CMakeLists.txt 文件，注释一行；
```

```
#include(/home/dji/opencv/build/OpenCVConfig.cmake)
```

```
#设置 OpenCV 路径为带有 GPU 加速的 opencv 4.6.0
```

```
include("~/opencv-4.6.0/build/OpenCVConfig.cmake")
```

```

19 #find_package(OpenCV REQUIRED)
20 #include(/home/dji/opencv/build/OpenCVConfig.cmake)
21 include("~/opencv-4.6.0/build/OpenCVConfig.cmake")
22 #message(WARNING "OpenCV_VERSION: ${OpenCV_VERSION}")
23
24 find_package(Ceres REQUIRED)
25

```

#修改 loop_fusion/CMakeLists.txt 文件,注释一行:

```
#include(/home/dji/opencv/build/OpenCVConfig.cmake)
```

#设置 OpenCV 路径为带有 GPU 加速的 opencv 4.6.0

```
include("~/opencv-4.6.0/build/OpenCVConfig.cmake")
```

```

18 #find_package(OpenCV)
19 #include(/home/dji/opencv/build/OpenCVConfig.cmake)
20
21 include("~/opencv-4.6.0/build/OpenCVConfig.cmake")
22 find_package(Ceres REQUIRED)
23
24 set(CMAKE_MODULE_PATH ${PROJECT_SOURCE_DIR}/cmake)
25 find_package(Eigen3)
26

```

此时直接编译会报错，需要修改版本兼容问题，按如下步骤操作即可编译成功：

```
cd ~/vins-fusion-gpu/src/VINS-Fusion-gpu
```

```
sed -i 's/CV_FONT_HERSHEY_SIMPLEX/cv::FONT_HERSHEY_SIMPLEX/g' `grep CV_FONT_HERSHEY_SIMPLEX -rl ./`
```

```
sed -i 's/CV_LOAD_IMAGE_GRAYSCALE/cv::IMREAD_GRAYSCALE/g' `grep CV_LOAD_IMAGE_GRAYSCALE -rl ./`
```

```
sed -i 's/CV_BGR2GRAY/cv::COLOR_BGR2GRAY/g' `grep CV_BGR2GRAY -rl ./`
```

```
sed -i 's/CV_RGB2GRAY/cv::COLOR_RGB2GRAY/g' `grep CV_RGB2GRAY -rl ./`
```

```
sed -i 's/CV_GRAY2RGB/cv::COLOR_GRAY2RGB/g' `grep CV_GRAY2RGB -rl ./`
```

```
sed -i 's/CV_GRAY2BGR/cv::COLOR_GRAY2BGR/g' `grep CV_GRAY2BGR -rl ./`
```

```
sed -i 's/CV_CALIB_CB_ADAPTIVE_THRESH/cv::CALIB_CB_ADAPTIVE_THRESH/g' `grep CV_CALIB_CB_ADAPTIVE_THRESH -rl ./`
```

```
sed -i 's/CV_CALIB_CB_NORMALIZE_IMAGE/cv::CALIB_CB_NORMALIZE_IMAGE/g' `grep CV_CALIB_CB_NORMALIZE_IMAGE -rl ./`
```

```
sed -i 's/CV_CALIB_CB_FILTER_QUADS/cv::CALIB_CB_FILTER_QUADS/g' `grep CV_CALIB_CB_FILTER_QUADS -rl ./`
```

```
sed -i 's/CV_CALIB_CB_FAST_CHECK/cv::CALIB_CB_FAST_CHECK/g' `grep CV_CALIB_CB_FAST_CHECK -rl ./`
```

```
sed -i 's/CV_ADAPTIVE_THRESH_MEAN_C/cv::ADAPTIVE_THRESH_MEAN_C/g' `grep CV_ADAPTIVE_THRESH_MEAN_C -rl ./`
```

```
sed -i 's/CV_THRESH_BINARY/cv::THRESH_BINARY/g' `grep CV_THRESH_BINARY -rl ./`
```

```
sed -i 's/CV_SHAPE_CROSS/cv::MORPH_CROSS/g' `grep CV_SHAPE_CROSS -rl ./`
```

```
sed -i 's/CV_SHAPE_RECT/cv::MORPH_RECT/g' `grep CV_SHAPE_RECT -rl .`

sed -i 's/CV_TERMCRIT_EPS/cv::TermCriteria::EPS/g' `grep CV_TERMCRIT_EPS -rl .`

sed -i 's/CV_TERMCRIT_ITER/cv::TermCriteria::MAX_ITER/g' `grep CV_TERMCRIT_ITER -rl .`

sed -i 's/CV_RETR_CCOMP/cv::RETR_CCOMP/g' `grep CV_RETR_CCOMP -rl .`

sed -i 's/CV_CHAIN_APPROX_SIMPLE/cv::CHAIN_APPROX_SIMPLE/g' `grep CV_CHAIN_APPROX_SIMPLE -rl .`

sed -i 's/CV_AA/cv::LINE_AA/g' `grep CV_AA -rl .`

sed -i 's/CV_LOAD_IMAGE_UNCHANGED/cv::IMREAD_UNCHANGED/g' `grep CV_LOAD_IMAGE_UNCHANGED -rl .`

sed -i 's/CV_MINMAX/cv::NORM_MINMAX/g' `grep CV_MINMAX -rl .`


cd ~/vins-fusion-gpu/

catkin_make
```

配置 ego-planner

安装必要的包：

```
sudo apt-get install libarmadillo-dev
```

```
sudo apt-get install ros-noetic-cv-bridge
```

```
sudo apt-get install ros-noetic-cmake-modules
```

直接复制幻思库中的 ego-planner 到 home 路径

编译以及运行（要在 vins-fusion 运行起来后在运行 ego-planner 工程）：

```
cd ego-planner
```

```
catkin_make -DCMAKE_BUILD_TYPE=Release
```

```
source devel/setup.bash
```

```
roslaunch ego_planner simple_run_vins.launch
```

全部工程自动运行

配置串口可执行权限：

用如下指令把权限配置文件复制到目录/etc/udev/rules.d/中

```
sudo cp 70-ttyusb.rules /etc/udev/rules.d/
```

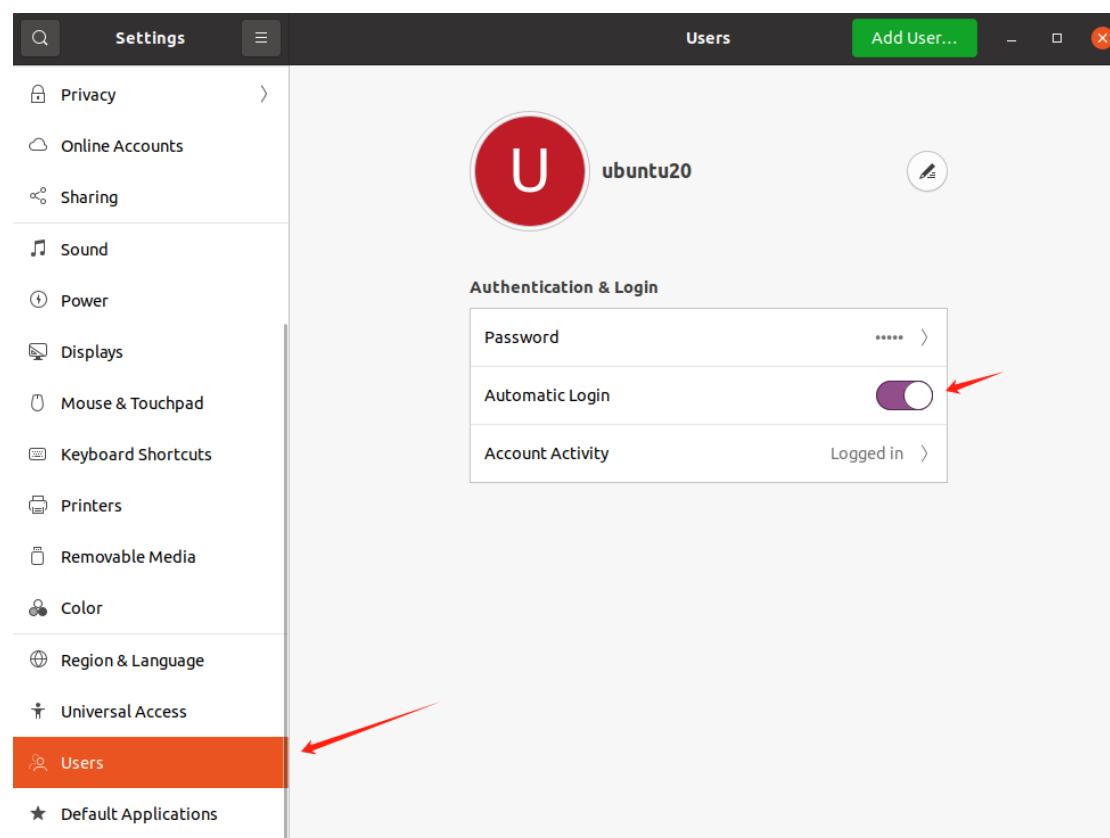
重启电脑后生效

```
sh auto_ap.sh
```

注意这个脚本的最后一段程序我们注释掉了，它是启动 vins 的 rviz 界面，注释掉的目的是如果允许这个 rviz 会影响 vins 实时性，只有静态测试时可以把注释掉的程序运行，如果在飞行过程中，这段程序必须注释掉禁止运行。

开机自启动

（1）配置 ubuntu 系统 auto login



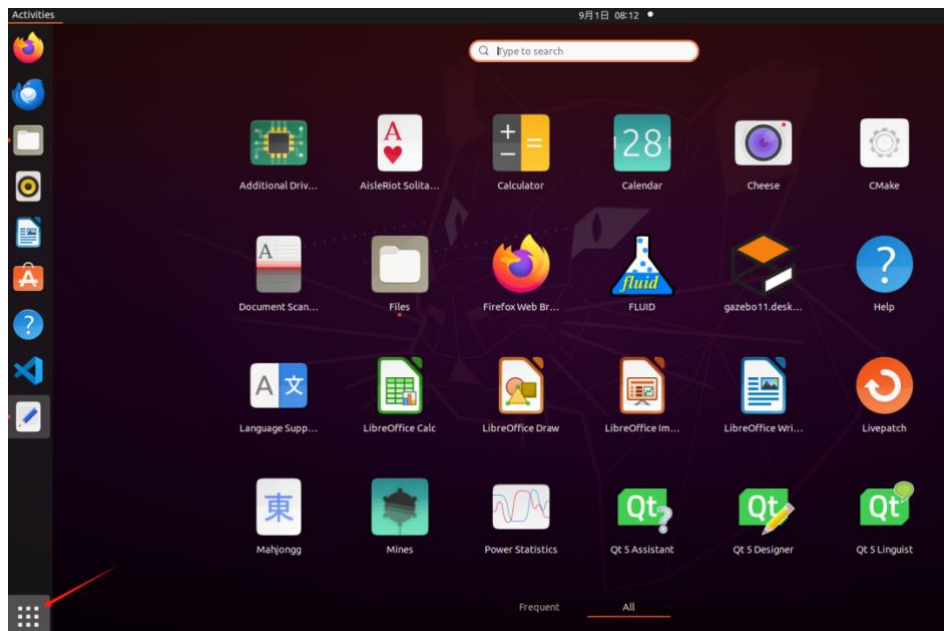
（2）配置自启动文件 auto_ap.sh

注意文件中的工程路径是否与自己的一致，不一致的自己修改一下。

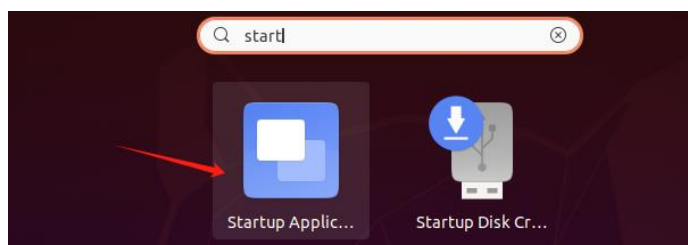
(3) 终端输入 `sh auto_ap.sh`

检测 `slam` 工程是否在一分钟内正常启动

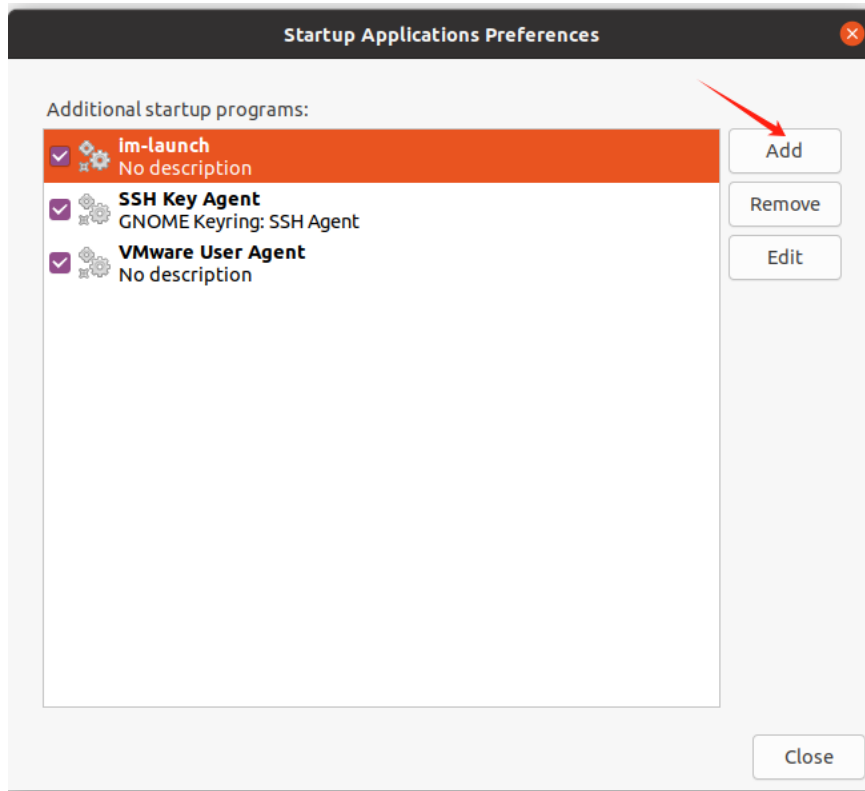
(4) 配置 `auto_ap.sh` 开机自动运行



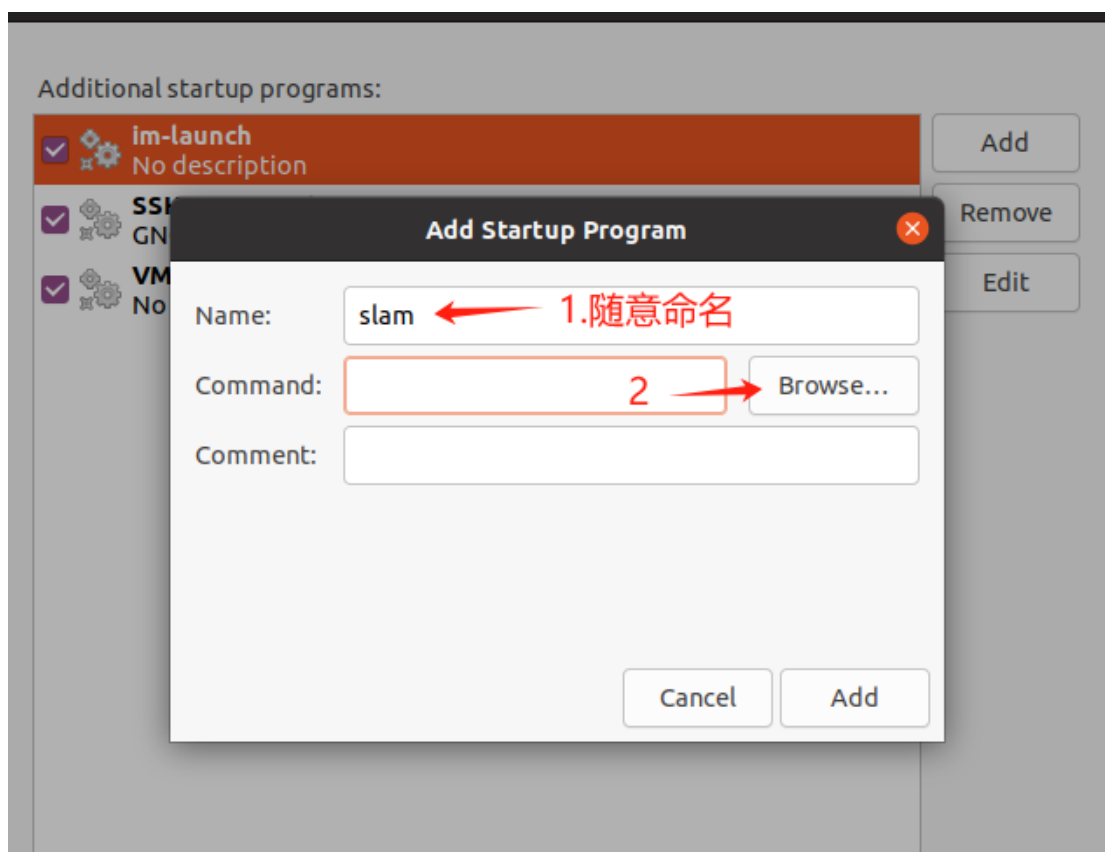
打开 Startup Application



点击 add



先填个 Name，再点 Browse 选择 auto_ap.sh 文件，选好后在“Command”栏路径前加上“sh ”（注意空格），例如：sh /xxxx/auto_ap.sh。



此时已完成开机自启动配置

（5）必须把 Jetson NX 的功率设置为 MAXN

重启系统验证一下吧！看看 slam 界面有没有在开机 1 分钟内正常运行。

注意到此为止不会出现点云地图，需要完成下一步中的飞控串口波特率设置后才会出现点云地图！

不启用 VINS（VSLAM）定位

注：

1. 是否启用 VINS 定位，请先联系幻思创新工程师对实际使用环境进行评估。
2. VINS 定位对环境要求较高，相机视野内需具备充足光照和丰富、稳定的纹理特征。若现场环境光照不足、纹理较少或特征不明显，则不建议启用 VINS 定位，以免影响定位效果。

当现场环境不满足 VINS 定位要求时，建议采用本方案，即关闭 VINS 定位，使用无人机下视光流定位作为替代方案。下视光流定位仅需保证飞行区域地面具有一定的纹理特征，即可实现稳定、可靠的定位。

本方案的配置步骤如下：

- （1）修改 `auto_ap.sh` 开机自启动脚本。

```
# !/bin/sh

sleep 30s
{
    gnome-terminal --tab "fcu" -- bash -c "source ~/ros_ws/devel/setup.bash;sleep 5s; roslaunch fcu_core fcu_core.launch; exec bash"
}&

sleep 10s
{
    gnome-terminal --tab "d435" -- bash -c "source ~/realsense_ws/devel/setup.bash;sleep 5s;roslaunch realsense2_camera rs_camera.launch;
}&

sleep 20s
{
    gnome-terminal --tab "vins" -- bash -c "source ~/vins-fusion-gpu/devel/setup.bash; sleep 5s; roslaunch vins realsense_d435i.launch; e
}&

sleep 20s
{
    gnome-terminal --tab "ego" -- bash -c "source ~/ego-planner/devel/setup.bash; sleep 5s; roslaunch ego_planner simple_run_vins.launch;
}&

# sleep 10s
# {
#   gnome-terminal --tab "vins" -- bash -c "source ~/vins-fusion-gpu/devel/setup.bash; sleep 5s; roslaunch vins vins_rviz.launch; exec ba
# }&
```

把框出来的部分（vins 的开机自启动程序）注释掉。

（2）烧录飞控固件到光流定位模式：

最新固件请参照[官网](#)—>“[下载中心](#)—>[FanciSwarm™ 固件及依赖包](#)”（[点击跳转](#)）。直接烧录官网下载的默认固件即可。

（3）App 配置波特率：

请按照[官网教程](#)—>“[用户手册](#)—>[FanciSwarm® 智能套件](#)—>[参数配置](#)”（[点击跳转](#)）将参数列表中的串口 1 波特率设置为 **460800**。

启用 VINS（VSLAM）定位（固件版本号≥V7.1.8-2025032001）

注：

1、要求无人机固件版本号高于或等于 V7.1.8-2025032001；

2、如果固件版本低于 V7.1.8-2025032001，请务必更新至最新版本；

3、最新固件请参照[官网](#)—>“[下载中心](#)—>[FanciSwarm™ 固件及依赖包](#)”（[点击跳转](#)）。

1、配置飞控固件参数

请参照[官网教程](#)—>“[高级开发指南](#)—>[工程开发指南](#)”（[点击跳转](#)），在固件工程 [Clibrary/include/config.h](#)（[点击跳转](#)）文件中，将以下 5 个参数配置为

下图所示的值（其中 USE_VINS 宏定义：使用 VINS 时设为 1，不使用时设为 0。此处需使用 VINS，因此配置为 1）。

特别注意：

（1）下图中的串口波特率（#define COMM_1_BANDRATE）需通过 App 进行配置，代码里配置无效！

（固件版本号≥V7.1.8-2025042201，App 版本号≥V1.3.4）

（2）App 配置波特率步骤：请按照[官网教程](#)→“[用户手册](#)→[FanciSwarm® 智能套件](#)→[参数配置](#)”（[点击跳转](#)）将参数列表中的串口 1 波特率设置为 460800。

```
74 #define COMM_0_MAV_COMM
75 #define COMM_1_MAV_COMM
76 #define COMM_2_LC302_COMM
77 #define COMM_3_GPS_COMM
78 #define COMM_4_MAV_COMM
79 #define COMM_UWB_DEV_COMM
80
81 // 串口波特率
82 #define COMM_1_BANDRATE 460800
83 #define COMM_2_BANDRATE 19200
84 #define COMM_3_BANDRATE 115200
85 #define COMM_4_BANDRATE 115200
86
87 // 配置 LED
88 #define FMU_LED_CONTROLL_ENABLE 1 // if use system default led control, set 1; if you want to control led by yourself, set 0;
89
90 // 配置磁罗盘
91 #define USE_MAG 0 // if use mag, set 1; if not use mag, set 0;
92
93 // 配置 GNSS
94 #define USE_GNSS 1 // if use gnss, set 1; if not use gnss, set 0;
95
96 // 配置 UWB
97 #define USE_UWB 1 // if use uwb, set 1; if don't use uwb, set 0;
98
99 // 配置光流
100 #define USE_FLOW 1 // if use optical flow, set 1; if don't use optical flow, set 0;
101
102 // 配置里程计
103 #define USE_ODOMETRY 1 // if use odometry, set 1; if don't use odometry, set 0;
104
105 // VINS
106 #define USE_VINS 1 // if use vins, set 1; if use lidar-slam, set 0;
107
```

注意！
最新版本固件的串口波特率需通过 App 进行配置，此处配置无效！

配置完成后，请务必重新编译。

2、烧录固件

请参照[官网教程](#)→“[高级开发指南](#)→[程序烧录指南](#)”（[点击跳转](#)）将上一步中编译生成的固件烧录到无人机中。

烧录固件后，我们重启机载电脑，由于已经配置好了开机自启动，fcu_core, vins, ego 会自动运行起来，如果我们在 vins 的 rviz 界面能看到特征点和轨迹，证明我们固件参数已经正确配置。

3、PID 调参

请按照[官网教程](#)—>“[用户手册](#)—>[FanciSwarm® 智能套件](#)—>[参数配置](#)”
[（点击跳转）](#)将参数列表中的 **Roll 速率 P** 调到 0.09，**Pitch 速率 P** 调到 0.09，
 如下图：

自稳Roll		自稳Pitch		自稳Yaw		高度		垂直速度	
P	4.5	P	4.5	P	4.5	P	1.0	P	5.0
Roll速率		Pitch速率		Yaw速率		垂直加速度			
P	0.09	P	0.09	P	0.18	P	0.5		
I	0.15	I	0.15	I	0.018	I	0.3		
D	0.0015	D	0.0015	D	0.0	D	0.0		
Imax	0.3	Imax	0.3	Imax	0.3	Imax	500.0		
Filt	20.0	Filt	20.0	Filt	2.5	Filt	20.0		

加速度计校准
 磁力计校准
 水平校准
 遥控器校准
 摇杆设置
 参数显示设置

4、VINS 调参

VINS-Fusion-gpu/config/realSense_d435i 文件夹中有三个 yaml 文件：

left.yaml

right.yaml

realSense_stereo_imu_config.yaml

其中，left.yaml 和 right.yaml 需要配置相机内参

```

1 %YAML:1.0
2 ---
3 model_type: PINHOLE
4 camera_name: camera
5 image_width: 640
6 image_height: 480
7 distortion_parameters:
8   k1: 0.0
9   k2: 0.0
10  p1: 0.0
11  p2: 0.0
12 projection_parameters:
13   fx: 380.5201416015625
14   fy: 380.5201416015625
15   cx: 318.6392822265625
16   cy: 241.96481323242188
  
```

fx=fy

内参通过相机话题的/camera/intra1/camera_info 和/camera/intra2/camera_info 查看

rostopic echo /camera/intra1/camera_info

rostopic echo /camera/intra2/camera_info

看其中的 K 矩阵，从前到后四个非零参数分别为 fx, cx, fy, cy;

一般两个摄像头的参数是相同的。

进一步配置 `realsense_stereo_imu_config.yaml`

```
3 #common parameters
4 #support: 1 imu 1 cam; 1 imu 2 cam; 2 cam;
5 imu: 1
6 num_of_cam: 2
7
8 imu_topic: "/imu_global_001"
9 image0_topic: "/camera/infra1/image_rect_raw"
10 image1_topic: "/camera/infra2/image_rect_raw"
11 output_path: "/home/nx/vins_output/"
12
13 cam0_calib: "left.yaml"
14 cam1_calib: "right.yaml"
15 image_width: 640
16 image_height: 480
17
18
19 # Extrinsic parameter between IMU and Camera.
20 estimate_extrinsic: 1 # 0 Have an accurate extrinsic param
21 # 1 Have an initial guess about extri
22
23
```

这里所有参数都要配置。

注意 `output_path` 后面的路径需要按自己 `output` 文件夹的实际路径填写！

下面需要标定相机外参。

```
26 body_T_cam0: !!opencv-matrix
27   rows: 4
28   cols: 4
29   dt: d
30   data: [ 9.9735272663772212e-01, -7.1619637167989844e-02,
31          -1.2576416032665960e-02, 8.3278634051004909e-03,
32          7.1847185491946020e-02, 9.9724064478573282e-01,
33          1.8683637874400087e-02, -2.2390599245467505e-02,
34          1.1203597867966783e-02, -1.9537757273067575e-02,
35          9.9974634554748520e-01, 3.2702141860980832e-02, 0., 0., 0., 1. ]
36 body_T_cam1: !!opencv-matrix
37   rows: 4
38   cols: 4
39   dt: d
40   data: [ 9.9751994449570225e-01, -7.0384127558861240e-02,
41          1.8687179835041469e-04, 5.9510495691064244e-02,
42          7.0367615029977015e-02, 9.9733682845829796e-01,
43          1.9174184617254526e-02, -1.8848869617343073e-02,
44          -1.5359323826330984e-03, -1.9113481852387801e-02,
45          9.9981614095952387e-01, 3.5348685186239563e-02, 0., 0., 0., 1. ]
46
```

首先在文件 `realsense_stereo_imu_config.yaml` 的 82 行配置一个存储路径，例如在 HOME 路径中建一个 `vins_output` 文件夹。注意我们工程可以通过 `auto_ap.sh` 脚本全部运行，运行后 `fcu_core` 终端会输出 `vins` 的轨迹坐标和偏航角。`vins` 标定方式是先在我们工程提供的默认的参数上按相机安装位置调整好矩阵初始参数，然后在启动 `vins` 工程后拿起飞机在光照和纹理良好的环境中走一圈。（注意走一圈过程中，相机镜头要朝向光照和纹理良好的开阔区域）如果走一圈回来以后 `vins` 没有发散，那么在 `vins_output` 文件夹中的 `extrinsic_parameter.csv` 文件中是最佳标定参数。把这个文件用文本编辑器打开，其中参数矩阵复制后填入 `realsense_stereo_imu_config.yaml`。

重复以上步骤，直到无人机在走一圈的前后的 `vins` 轨迹没有明显偏差，此时的参数就是优化到最佳的参数。

其它参数保持我们工程文件中的默认配置无需修改。

```
51 #Multiple thread support
52 multiple_thread: 1
53 use_gpu: 1
54 use_gpu_acc_flow: 1
55
56 #feature traker parameters
57 max_cnt: 100 # max feature number in feature tracking
58 min_dist: 30 # min distance between two features
59 freq: 10 # frequency (Hz) of publish tracking result. At least 10Hz for good estimation. If set 0, the frequency will be same as raw image
60 F_threshold: 1.0 # ransac threshold (pixel)
61 show_track: 0 # publish tracking image as topic
62 flow_back: 1 # perform forward and backward optical flow to improve feature tracking accuracy
63
64 #optimization parameters
65 max_solver_time: 0.04 # max solver iteration time (ms), to guarantee real time
66 max_num_iterations: 8 # max solver iterations, to guarantee real time
67 keyframe_parallax: 10.0 # keyframe selection threshold (pixel)
68
69 #imu parameters The more accurate parameters you provide, the better performance
70 acc_n: 0.2 # accelerometer measurement noise standard deviation. #0.2
71 gyr_n: 0.05 # gyroscope measurement noise standard deviation. #0.05
72 acc_w: 0.02 # accelerometer bias random work noise standard deviation. #0.02
73 gyr_w: 0.002 # gyroscope bias random work noise standard deviation. #4.0e-5
74 g_norm: 9.805 # gravity magnitude
75
76 #unsynchronization parameters
77 estimate_td: 1 # online estimate time offset between camera and imu
78 td: 0.0 # initial value of time offset. unit: s. readed image clock + td = real image clock (IMU clock)
79
80 #loop closure parameters
81 load_previous_pose_graph: 0 # load and reuse previous pose graph; load from 'pose_graph_save_path'
82 pose_graph_save_path: "/home/nv/vins_output/" # save and load path
83 save_image: 1 # save image in pose graph for visualization purpose; you can close this function by setting 0
```

use_gpu 和 use_gpu_acc_flow 都为 1 才能实现最佳 GPU 加速

注意 pose_graph_save_path 后面的路径需要按自己 output 文件夹的实际路径填写！

VINS 调参结束后，我们用无人机的电池供电，启动无人机，连接显示器，等待所有机载电脑工程运行起来。拿起飞机在房间中远距离走几圈后，将机载电脑重新连接显示屏，检查 fcu_core 工程终端输出的 VINS 原始坐标和 RViz 界面中的轨迹坐标是一致的。如果不一致，说明配置得不对，可以联系幻思创新的工程师寻求协助。

特别注意：

刚启动 VINS 要缓慢晃一晃使其完成初始化，看 fcu_core 工程输出的 vins 坐标飘不飘，飘了就重新启动。

5、飞行测试（测试定位源）

（1）有专业飞手（姿态模式起飞）

特别注意！使用遥控器操控前请务必仔细阅读“FanciSwarm® 飞行前检查”一节。

姿态模式是一种对操控者要求较高的无人机飞行模式，适用于有一定飞行经验的用户。在此模式下，遥控器 CH7 三段开关需要调至 **上档位** 以启用该功能。

如果操控者飞行经验不足，严禁使用姿态模式进行飞行，以免因操作不当导致无人机失控。

请参照 [官网教程](#)—>“快速入门指南—>[FanciSwarm/ FanciSwarm Pro 飞行](#)—>使用遥控器飞行—>特别说明 2”（点击跳转）。

（注：只需按照①和②操作，③在当前阶段无需执行，请跳过。）

特别说明2

（1）CH5三段开关和CH7三段开关档位详细说明，如下图所示：



（2）使用遥控器调试需要使用左边（ch7）三段开关，其中三个档位从上到下分别是姿态模式、定点、ros用或者机载电脑用。调试流程可参照下述步骤。（调试环节，切勿掉以轻心，务必安全操作）

- ① 如果想测定位源是不是好用，首先拨动ch7三段开关使无人机在姿态模式下起飞；
- ② 然后，在空中切到中间档位（定点），用来测试定位源飘不飘，这种情况下可以随时切回到姿态模式；

（2）无专业飞手（定点模式起飞）

特别注意！使用遥控器操控前请务必仔细阅读“FanciSwarm® 飞行前检查”一节。

定点模式，即遥控器 CH7 三段开关调到中间档位。

请参照 [官网教程](#)—>“快速入门指南—>[FanciSwarm/ FanciSwarm Pro 飞行](#)—>使用遥控器飞行”（点击跳转）。

（注：在定点模式下，具有飞行经验的用户可随时将 **CH7** 三段开关切换至上档位，进入姿态模式，以便在测试过程中及时控制无人机，防止因异常情况导致失控。）

EGO 调参

在 planner/plan_manage/launch 文件夹中调两个文件

simple_run_vins.launch

odom 话题，相机话题，相机内参，最大速度，最大加速度是关键参数。

```

1 <launch>
2 <!-- size of map, change the size inflate x, y, z according to your application -->
3 <arg name="map_size_x" value="40.0"/>
4 <arg name="map_size_y" value="40.0"/>
5 <arg name="map_size_z" value="3.0"/>
6
7 <!-- topic of your odometry such as VIO or LIO -->
8 <arg name="odom_topic" value="/odom_global_001" />
9
10 <!-- main algorithm params -->
11 <include file="$(find ego_planner)/launch/advanced_param.xml">
12
13   <arg name="map_size_x" value="$(arg map_size_x)"/>
14   <arg name="map_size_y" value="$(arg map_size_y)"/>
15   <arg name="map_size_z" value="$(arg map_size_z)"/>
16   <arg name="odometry_topic" value="$(arg odom_topic)"/>
17
18   <!-- camera pose: transform of camera frame in the world frame -->
19   <!-- depth topic: depth image, 640x480 by default -->
20   <!-- don't set cloud_topic if you already set these ones! -->
21   <arg name="camera_pose_topic" value="/vins_estimator/camera_pose"/>
22   <arg name="depth_topic" value="/camera/depth/image_rect_raw"/>
23
24   <!-- topic of point cloud measurement, such as from LIDAR -->
25   <!-- don't set camera pose and depth, if you already set this one! -->
26   <arg name="cloud_topic" value="/pcl_render_node/cloud"/>
27
28   <!-- intrinsic params of the depth camera -->
29   <arg name="cx" value="321.04638671875"/>
30   <arg name="cy" value="243.44969177246094"/>
31   <arg name="fx" value="387.229248046875"/>
32   <arg name="fy" value="387.229248046875"/>
33
34   <!-- maximum velocity and acceleration the drone will reach -->
35   <arg name="max_vel" value="0.3" />
36   <arg name="max_acc" value="0.5" />
37
38   <!-- always set to 1.5 times greater than sensing horizon -->
39   <arg name="planning_horizon" value="7.5" />
40
41   <!-- 1: use 2D Nav Goal to select goal -->
42   <!-- 2: use global waypoints below -->
43   <arg name="flight_type" value="1" />
44

```

advanced_param.xml

膨胀半径 obstacles_inflation 按照飞机尺寸设置

```

72 <param name="grid_map/resolution" value="0.1" />
73 <param name="grid_map/map_size_x" value="$(arg map_size_x)" />
74 <param name="grid_map/map_size_y" value="$(arg map_size_y)" />
75 <param name="grid_map/map_size_z" value="$(arg map_size_z)" />
76 <param name="grid_map/local_update_range_x" value="5.5" />
77 <param name="grid_map/local_update_range_y" value="5.5" />
78 <param name="grid_map/local_update_range_z" value="4.5" />
79 <param name="grid_map/obstacles_inflation" value="0.099" />
80 <param name="grid_map/local_map_margin" value="30" />
81 <param name="grid_map/ground_height" value="-0.01" />

```

虚拟天花板 virtual_ceil_height 设置为 1.0 米，不让飞机飞太高

```

104 <param name="grid_map/virtual_ceil_height" value="1.0" />
105 <param name="grid_map/visualization_truncate_height" value="2.5" />
106 <param name="grid_map/show_occ_time" value="false" />
107 <param name="grid_map/pose_type" value="2" />
108 <param name="grid_map/frame_id" value="world" />
109

```

启用 EGO 和下发目标点

1、说明

启用 EGO 和下发目标点的操作共分为 17 个步骤。请严格按照以下步骤执行操作。其中，**下发目标点的步骤需使用个人电脑完成。**

2、飞行测试

为了方便操作和查看无人机的状态，建议用户使用遥控器进行飞行控制，同时通过 FanciSwarm Pro App 查看无人机实时状态数据，以提升飞行体验和安全性。

请参照 [官网教程](#)—>“快速入门指南”—>[FanciSwarm/ FanciSwarm Pro 飞行](#)—>[使用遥控器飞行](#)”（[点击跳转](#)）。**特别注意！使用遥控器操控前请务必仔细阅读“FanciSwarm® 飞行前检查”一节。**

只有在确认无人机飞行稳定后，方可继续进行下一步。

3、启用 VINS（VSLAM）定位（若已启用，请跳过此步骤）

按照上述章节“**启用 VINS（VSLAM）定位**”的 1、2、3、4 步骤和“**EGO 调参**”进行操作。

请确保无人机飞行状态稳定后，再继续执行下一步操作。

4、配置组网模块（飞多机集群需要配置，飞单机无需配置）

该步骤确定的 IP 地址将在 **7、配置通信节点参数** 中使用。

（1）如果仅飞一架飞机，无需进行额外配置。在后续步骤中，可直接使用个人电脑连接飞机的**默认 IP 地址：192.168.4.1**。

（2）如果飞多架飞机（飞集群），请参照[官网教程](#)—>“[集群开发指南](#)”—>[③ 配置组网模块](#)”（[点击跳转](#)）。

5、配置个人电脑环境（若已配置完成，请跳过此步骤）

该步骤用于配置 **个人电脑（远程电脑）环境**，而非无人机的机载电脑。请注意区分两者，避免误操作。

配置教程请参照[官网教程](#)—>“[集群开发指南](#)—>④ 配置电脑环境”（[点击跳转](#)）。

6、导入 ROS 工程

请参照[官网教程](#)—>“[集群开发指南](#)—>⑤ 导入 ROS 工程”（[点击跳转](#)）。

7、配置通信节点参数

（1）飞单机

首先，打开刚导入的 ROS 工程（fcu_core 文件）的 [fcu_bridge_001.cpp](#)（[点击跳转](#)）文件。在文件顶部可以看到参数配置变量，请将这些变量按照下图所示进行配置：

```
#define BUF_SIZE 32768//数据缓存区大小
#define BAUDRATE 460800 //虚拟串口波特率
#define DRONE_PORT 333 //port
static char* DRONE_IP = "192.168.0.201"; //ip
static char* USB_PORT = "/dev/ttyACM0"; //usb虚拟串口文件描述符
static mavlink_channel_t mav_chan=MAVLINK_COMM_1;//MAVLINK_COMM_0虚拟串口发送，MAVLINK_COMM_1网口发送
static bool offboard=false;//是否使用机载电脑
static bool use_uwb=false;//是否使用UWB基站
static bool set_goal=true;//远程电脑用于设置轨迹规划的目标，机载电脑应为false
static bool simple_target=true;//仅机载电脑配置：是否为简单目标点，simple_target表示目标只有位置，没有速度和加速度
```

其中，根据 **4、配置组网模块——飞单机** 的说明，参数“DRONE_IP”应填写为“192.168.4.1”，即飞机的默认 IP 地址。

（2）多无人机通信节点参数配置说明（以三机集群协同为例）

同时飞 3 架飞机，需要运行 3 个通信节点：

- （a）通信节点 001（对应于 ROS 工程中的 fcu_bridge_001.cpp 文件）；
- （b）通信节点 002（对应于 ROS 工程中的 fcu_bridge_002.cpp 文件）；
- （c）通信节点 003（对应于 ROS 工程中的 fcu_bridge_003.cpp 文件）。

因此需要对 3 个通信节点参数进行配置。

以配置通信节点 001（`fcu_bridge_001.cpp` 文件）为例进行讲解，配置过程请参照 **（1）飞单机** 中的步骤，其中参数“`DRONE_IP`”应填入 **4、配置组网模块——飞多架飞机** 中配置的组网模块的 IP 地址。

注意：

`fcu_bridge_001.cpp`、`fcu_bridge_002.cpp`、`fcu_bridge_003.cpp` 中的参数“`DRONE_IP`”为 FanciSwarm 无人机网络通信的 IP，用户需要填入 **4、配置组网模块——飞多架飞机** 中配置的组网模块的 IP 地址，三个通信节点（3 台无人机）对应三个 ip 地址，请一一对应。

8、目标点设定与开发

目标点在 ROS 工程 [fcu_mission.cpp](#)（[点击跳转](#)）文件中定义。

特别说明：

- （1）坐标系的原点为机载电脑 slam 启动时的位置点；
- （2）坐标系的 X 轴方向是 slam 启动时的机头朝向；
- （3）坐标系的 Y 轴方向 X 轴方向右侧的 90 度；
- （4）坐标系的 Z 轴方向垂直向下，为了和高度方向统一，特意将目标点的 Z 坐标设置为高度。

（1）飞单机

在 [fcu_mission.cpp](#)（[点击跳转](#)）文件中的 `void execute_mission_001(void)` 函数内，用户可以根据需求定义目标点的坐标（单位：米）。下图展示了示例中包含的 4 个目标点的坐标值，用户可根据实际需求进行自定义配置。

```

101     switch(enable_pos){
102     case 1:
103         px=0.0;
104         py=0.0;
105         pz=1.0;
106
107         px1=pos_takeoff_001_x+px; py1=pos_takeoff_001_y+py; pz1=pz;
108         px2=pos_takeoff_002_x+px; py2=pos_takeoff_002_y+py; pz2=pz;
109         px3=pos_takeoff_003_x+px; py3=pos_takeoff_003_y+py; pz3=pz;
110         px4=pos_takeoff_004_x+px; py4=pos_takeoff_004_y+py; pz4=pz;
111         px5=pos_takeoff_005_x+px; py5=pos_takeoff_005_y+py; pz5=pz;
112         px6=pos_takeoff_006_x+px; py6=pos_takeoff_006_y+py; pz6=pz;
113
114         break;
115     case 2:
116         px=0.5;
117         py=0.0;
118         pz=1.0;
119
120         px1=pos_takeoff_001_x+px; py1=pos_takeoff_001_y+py; pz1=pz;
121         px2=pos_takeoff_002_x+px; py2=pos_takeoff_002_y+py; pz2=pz;
122         px3=pos_takeoff_003_x+px; py3=pos_takeoff_003_y+py; pz3=pz;
123         px4=pos_takeoff_004_x+px; py4=pos_takeoff_004_y+py; pz4=pz;
124         px5=pos_takeoff_005_x+px; py5=pos_takeoff_005_y+py; pz5=pz;
125         px6=pos_takeoff_006_x+px; py6=pos_takeoff_006_y+py; pz6=pz;
126
127         break;
128     case 3:
129         px=2.0;
130         py=2.0;
131         pz=0.5;
132
133         px1=pos_takeoff_001_x+px; py1=pos_takeoff_001_y+py; pz1=pz;
134         px2=pos_takeoff_002_x+px; py2=pos_takeoff_002_y+py; pz2=pz;
135         px3=pos_takeoff_003_x+px; py3=pos_takeoff_003_y+py; pz3=pz;
136         px4=pos_takeoff_004_x+px; py4=pos_takeoff_004_y+py; pz4=pz;
137         px5=pos_takeoff_005_x+px; py5=pos_takeoff_005_y+py; pz5=pz;
138         px6=pos_takeoff_006_x+px; py6=pos_takeoff_006_y+py; pz6=pz;
139
140         break;
141     case 4:
142         px=0.0;
143         py=0.5;
144         pz=1.0;
145
146         px1=pos_takeoff_001_x+px; py1=pos_takeoff_001_y+py; pz1=pz;
147         px2=pos_takeoff_002_x+px; py2=pos_takeoff_002_y+py; pz2=pz;
148         px3=pos_takeoff_003_x+px; py3=pos_takeoff_003_y+py; pz3=pz;
149         px4=pos_takeoff_004_x+px; py4=pos_takeoff_004_y+py; pz4=pz;
150         px5=pos_takeoff_005_x+px; py5=pos_takeoff_005_y+py; pz5=pz;
151         px6=pos_takeoff_006_x+px; py6=pos_takeoff_006_y+py; pz6=pz;
152
153         break;
154     default:
155         px1=pos_takeoff_001_x; py1=pos_takeoff_001_y; pz1=0.0f;
156         px2=pos_takeoff_002_x; py2=pos_takeoff_002_y; pz2=0.0f;
157         px3=pos_takeoff_003_x; py3=pos_takeoff_003_y; pz3=0.0f;
158         px4=pos_takeoff_004_x; py4=pos_takeoff_004_y; pz4=0.0f;
159         px5=pos_takeoff_005_x; py5=pos_takeoff_005_y; pz5=0.0f;
160         px6=pos_takeoff_006_x; py6=pos_takeoff_006_y; pz6=0.0f;
161         break;

```

（2）多无人机目标点配置说明（以三机集群协同为例）

在多无人机任务中，用户可以通过设定各通信节点的目标点来实现协同飞行。

任务代码包含以下三个通信节点和对应函数：

- 通信节点 001（1 号无人机）：`void execute_mission_001(void)`
- 通信节点 002（2 号无人机）：`void execute_mission_002(void)`
- 通信节点 003（3 号无人机）：`void execute_mission_003(void)`

用户只需参考 `void execute_mission_001(void)` 的代码结构和目标点设定逻辑，在 `void execute_mission_002(void)` 和 `void execute_mission_003(void)` 中实现类似的目标点配置即可。

9、编译 ROS 工程

请参照[官网教程](#)—>“[集群开发指南](#)—>⑧ 编译 ROS 工程”（[点击跳转](#)）。

10、开启无人机

启动无人机后，将其放置在水平面上，确保机尾朝向操控者。

11、运行 ROS 工程

请参照[官网教程](#)—>“[集群开发指南](#)—>⑬ 运行 ROS 工程”（[点击跳转](#)）。

12、检查轨迹

拿起飞机走一圈，并通过 RViz 界面检查轨迹的正确性。在确认轨迹显示无误后，方可进入后续操作步骤。

13、清除 RViz 界面轨迹

请参照[官网教程](#)—>“[集群开发指南](#)—>⑰ 清除 RViz 界面轨迹”（[点击跳转](#)）。

14、飞行测试（通过键盘一键下发指令）

使用键盘输入指令字符下发控制指令，分别测试每架无人机的“一键起飞”和“一键降落”功能。请按照以下步骤进行操作。

第一步：无人机解锁

在解锁前，特别注意以下事项：

（1）在无人机启动提示音结束后，将飞机垂直拿起（拿起一次即可），**拿到距离地面超过 1m 后放下**。如果激光测距工作正常，解锁才能顺利进行，否则 FanciSwarm 会发出“滴滴滴...”的报警音，同时，固件强制锁定，不允许起飞。

（2）FanciSwarm App 与 ROS 不能同时连接无人机，**同一时刻 App 与 ROS 只能连一个**；

（3）**遥控器和 ROS 可同时连接无人机，建议保持两者连接，以便在测试过程中异常时迅速切换为遥控器控制，防止失控**。在同时连接的情况下，若用户想通过 **ROS 控制无人机**，则需将遥控器的控制模式切换至**自主模式档位（ROS 用或者机载电脑用）**；相反，若用户想直接使用**遥控器来控制无人机**，则需将遥控器的控制模式调整至**位置模式档位（定点）**。



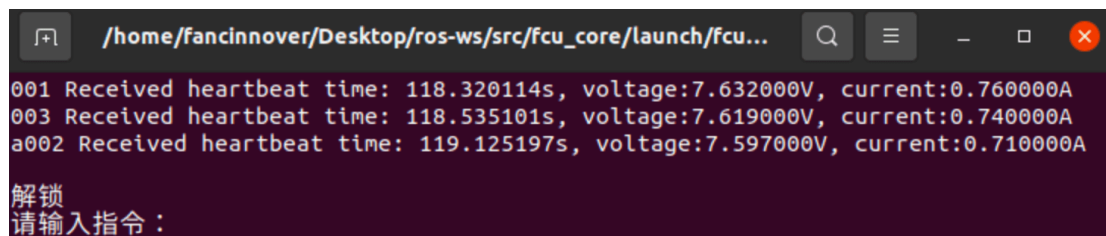
（4）遥控器油门始终有效，当油门低于 0.4 时，飞机会下降；当油门高于 0.6 时，飞机会上升；当油门在 0.4-0.6 之间时，飞机会保持悬停状态。**若用户希望无人机的飞行的高度由 ROS 控制，则需确保遥控器的油门持续保持在 0.4 至 0.6 的范围内，以此避免遥控器油门对飞机高度控制的干扰。**

完成上述检查后：

（1）确保已运行 ROS 工程；

（2）在连接遥控器的情况下，并确保遥控器的控制模式切换至**自主模式档位（ROS 用或者机载电脑用）**。

在终端输入字母“a”，然后按下“回车键”（无需在输入“a”后立即按回车）。终端界面将显示“解锁”字样。如下图所示：

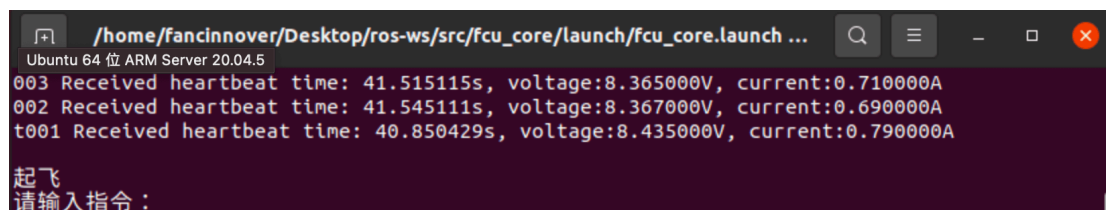


```
/home/fancinover/Desktop/ros-ws/src/fcu_core/launch/fcu...
001 Received heartbeat time: 118.320114s, voltage:7.632000V, current:0.760000A
003 Received heartbeat time: 118.535101s, voltage:7.619000V, current:0.740000A
a002 Received heartbeat time: 119.125197s, voltage:7.597000V, current:0.710000A
解锁
请输入指令：
```

此时，无人机启动，并伴随着启动声音，无人机完成解锁。

第二步：一键起飞

在终端输入“t”，然后按下“回车键”（无需在输入“t”后立即按回车）。终端界面将显示“起飞”字样，如下图所示：

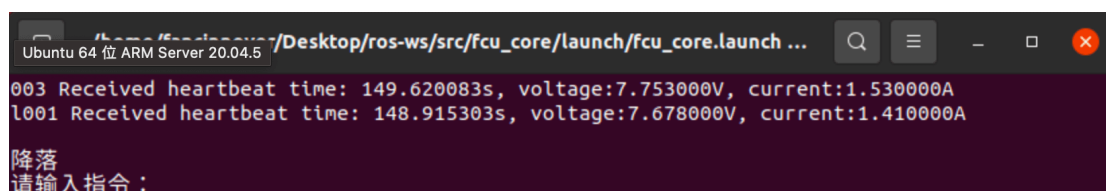


```
/home/fancinover/Desktop/ros-ws/src/fcu_core/launch/fcu_core.launch ...
Ubuntu 64 位 ARM Server 20.04.5
003 Received heartbeat time: 41.515115s, voltage:8.365000V, current:0.710000A
002 Received heartbeat time: 41.545111s, voltage:8.367000V, current:0.690000A
t001 Received heartbeat time: 40.850429s, voltage:8.435000V, current:0.790000A
起飞
请输入指令：
```

此时，FanciSwarm® 无人机起飞到一定高度（起飞高度可在 FanciSwarm App 中设置）并悬停。

第三步：一键降落

在终端输入“l”（小写的 L），然后按下“回车键”（无需在输入“l”后立即按回车）。终端界面将显示“降落”字样，如下图所示：



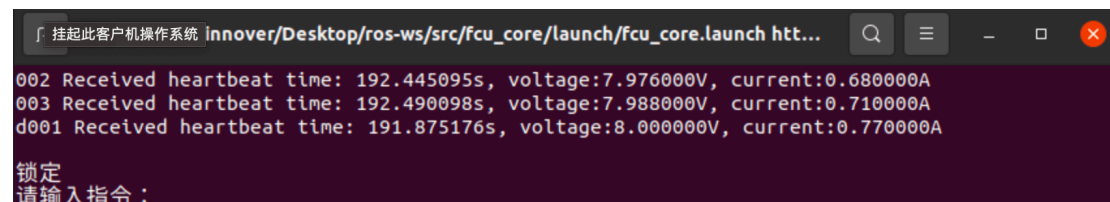
```
/home/fancinover/Desktop/ros-ws/src/fcu_core/launch/fcu_core.launch ...
Ubuntu 64 位 ARM Server 20.04.5
003 Received heartbeat time: 149.620083s, voltage:7.753000V, current:1.530000A
l001 Received heartbeat time: 148.915303s, voltage:7.678000V, current:1.410000A
降落
请输入指令：
```

此时，FanciSwarm® 无人机开始降落，并着陆。

第四步：无人机锁定

特别注意：一键锁定为强制指令，飞行过程中如触发该指令，将导致无人机立即坠落。请谨慎操作！

等待 FanciSwarm® 无人机着陆后，在终端输入“d”，然后按下“回车键”（无需输入“d”后立即按回车）。终端界面将显示“锁定”字样，如下图所示：



```
Innover/Desktop/ros-ws/src/fcu_core/launch/fcu_core.launch htt...
002 Received heartbeat time: 192.445095s, voltage:7.976000V, current:0.680000A
003 Received heartbeat time: 192.490098s, voltage:7.988000V, current:0.710000A
d001 Received heartbeat time: 191.875176s, voltage:8.000000V, current:0.770000A
锁定
请输入指令：
```

请确保此步骤顺利完成后，再继续执行下一步操作。

15、下发目标点

通过键盘输入指令字符下发目标点，请依照以下步骤操作。

第一步：无人机解锁

请参照 上一节——**14、飞行测试**。

第二步：一键起飞

请参照 上一节——**14、飞行测试**。

第三步：下方发目标点

在终端输入“1”（数字1），然后按下“回车键”（无需在输入“1”后立即按回车），即可完成下发 **8、目标点设定与开发** 中的第1个目标点。

同理，在终端输入“4”（数字4），然后按下“回车键”（无需在输入“4”后立即按回车），即可完成下发 **8、目标点设定与开发** 中的第4个目标点。

第四步：一键降落

请参照 上一节——**14、飞行测试**。

第五步：无人机锁定

请参照 上一节——**14、飞行测试**。

特别说明：

所有指令字符均定义在 [fcu_command.cpp](#)（[点击跳转](#)）中定义，用户可参考源码进行自定义操作。

16、退出 ROS 工程

请参照[官网教程](#)—>“[集群开发指南](#)—> ②1 退出 ROS 工程”（[点击跳转](#)）。

17、关闭无人机

FanciSwarm® 无人机锁定后，如果不继续飞行，请及时关闭每台无人机并拔掉电池，以避免电量损耗，节省电池电量。

至此，“启用 EGO 并下发目标点”的飞行过程已全部完成。